

AI Engineering Techniques and Architectures Project



Project Overview

For this project, you will design, build, deploy, and evaluate an agentic AI system that helps users complete HR policy and operations tasks for a hypothetical company. The system must include a Retrieval-Augmented Generation (RAG) capability over a corpus of internal company policy and procedure documents. It must also include an agentic layer that can plan, select tools, call one or more Model Context Protocol (MCP) servers, use mock structured data such as employee records and PTO or benefits balances, and produce grounded, cited responses.

The completed application should be deployed to a free-tier host such as Render, Railway, or an equivalent platform. To remain free-tier compatible, students may run the web application, agent orchestrator, RAG index, and MCP server processes within a single deployed service, or they may deploy MCP servers as separate services if their hosting plan permits. The agentic system must also run locally for development.

Finally, you will demonstrate the system via a recorded screen-share video showing the deployed application carrying out two agentic tasks. During the demo, the presenter must explain how the agent is correctly calling MCP-exposed tools, including the relevant tool names, tool-call arguments, returned results, retrieved citations, and final response/ behavior.

You can complete this project either individually or as a group of no more than three people. While you can fully hand code this project if you wish, you are strongly encouraged to use leading AI code generation tools such as Claude Code, Codex, Cursor or Antigravity or leading models to assist in rapidly producing your solution, being sure to describe in broad terms how you used them. You will be graded on the quality and functionality of the application and how well it meets the project requirements; no given proportion of the code is required to be hand coded.

Learning Outcomes

- Demonstrate excellent AI engineering skills through the design, implementation, deployment, and evaluation of an agentic AI system.
- Demonstrate the ability to select appropriate AI system design patterns and architectures, including RAG, tool use, agent orchestration, and MCP-based integration.
- Implement a working LLM-based agentic system that combines policy RAG with agentic workflows over structured mock data.
- Build and call MCP servers that expose tools used by the AI agent to complete realistic HR tasks.
- Evaluate both answer quality and agent behavior, including groundedness, citation accuracy, tool selection, workflow completion, and latency.
- Utilize AI tooling as appropriate and document the impact of that tooling on the development process.

Project Description

First, assemble a small but coherent corpus of documents outlining company policies and procedures - about 5-20 short markdown, HTML, PDF, or TXT files totaling 30-120 pages. The corpus should support realistic employee questions about topics such as PTO, holidays, remote work, expenses, data security, benefits, onboarding, equipment, leave, and workplace conduct. You may author the documents yourself with AI assistance or use documents that you are legally allowed to include in the repository or load at runtime. No private or paid data is required.

In addition, create small mock structured datasets in JSON, CSV, or other. These may include employee profiles, PTO balances, benefits elections, manager relationships, office locations, employment type, or ticket records. Mock data should be clearly synthetic and should not contain real private employee information.

Use free or zero-cost options where possible. You may use free-tier AI model providers, OpenRouter, Groq, local models, or your own API keys. For embedding models, free-tier options are available from several providers, and local embedding models are acceptable. Students should design the system so it can run on modest free-tier resources by keeping the corpus and mock datasets relatively small.

1. Environment and Reproducibility

- Create a virtual environment, such as venv, conda, or an equivalent setup.
- List dependencies in requirements.txt, pyproject.toml, package.json, or environment.yml as appropriate.
- Provide a README.md with setup, local run, deployment, and evaluation instructions.
- Set fixed seeds where applicable, such as deterministic chunking or evaluation sampling.
- Ensure secrets such as API keys are read from environment variables and are not committed to the repository.

2. Policy Corpus Ingestion and Indexing

- Parse and clean policy documents, handling at least two supported source formats where feasible, such as markdown, HTML, PDF, or TXT.
- Chunk documents using a justified strategy, such as heading-aware chunking or token windows with overlap.
- Embed chunks using a free embedding model, local model, or free-tier API.
- Store embedded chunks in a local or lightweight vector database such as Chroma, FAISS, PostgreSQL with vector extension, or a hosted vector store if available.
- Persist enough document metadata to support citations, including document title or ID, section, and source snippet.

3. Retrieval Augmented Generation (RAG)

- Implement top-k retrieval with optional filtering, query rewriting, or reranking.
- Build a prompting strategy that injects retrieved chunks and source metadata into the LLM context.
- Generate answers that cite source document IDs, titles, or sections and include supporting snippets where appropriate.
- Add guardrails that refuse or redirect out-of-corpus policy questions, limit unsupported claims, and distinguish policy facts from recommendations.
- Include at least one complex question requiring retrieval from multiple policy documents.

4. Agentic System Design

- Build an agent orchestrator that can interpret user intent, decide whether RAG alone is sufficient, select tools, call MCP-exposed tools, and synthesize final responses.
- Support at least two multi-step HR workflows, such as remote work eligibility, PTO request guidance, benefits question handling, expense compliance, onboarding checklist creation, or HR case triage.
- Implement a visible or logged trace of agent reasoning steps at the architectural level: selected tools, tool arguments, tool outputs, retrieved policy sources, final answer basis, and any escalation decision. Do not expose hidden chain-of-thought; provide concise operational traces instead.
- Handle failures gracefully, such as unavailable MCP tools, missing employee IDs, incomplete policy evidence, or ambiguous requests that require clarification.
- Prevent irreversible actions. Actions such as creating HR tickets, drafting manager messages, or updating case records must be mock actions or require explicit user confirmation.

5. MCP Server and Tool Integration

- Implement one or more MCP servers that expose tools for the agent. The MCP server may run as a local process using stdio, a localhost HTTP service, a Streamable HTTP service, or another MCP-compatible approach supported by your system's architecture.
- Expose at least five MCP tools. At least one tool must use the RAG index or retrieve policy evidence, and at least one tool must use mock structured data or perform a mock operation.
- Tools should include tools such as the following: `search_policy_documents`, `get_policy_section`, `lookup_employee_profile`, `check_pto_balance`, `lookup_benefits_status`, `create_mock_hr_ticket`, `draft_hr_email`, and `check_policy_compliance`.
- The agent must actually call MCP-exposed tools during execution; hard-coded direct function calls are not sufficient unless they are wrapped and invoked through the MCP layer.
- Document the MCP architecture, transport choice, tool schemas, and how the agent client discovers and calls tools.

6. Web Application

- Students can use Flask, FastAPI, Streamlit, Node/Express, Next.js, or an alternative framework for the web application.
- The web app should include a chat interface where users can ask HR policy and workflow questions.
- Provide a `/chat` endpoint, or equivalent, that receives user requests and returns the final answer, citations, snippets, and a concise tool-call trace.
- Provide a `/health` endpoint, or equivalent, that returns a simple JSON status including app status and, where feasible, MCP connectivity status.
- Provide a way for the grader to reproduce at least two agentic demo tasks from the UI or an API client.

7. Deployment to Render, Railway, or Equivalent

- Deploy the application to Render, Railway, or an equivalent free-tier or zero-cost host. The deployed application must be accessible at a shareable URL unless the student documents a platform outage or exceptional deployment issue.
- To remain free-tier compatible, a single-service deployment is acceptable: the web app, agent orchestrator, local vector store, mock JSON data, and MCP server process may all run in one deployed service.
- Students with sufficient free-tier resources are encouraged to deploy the MCP server as a separate service and call it via HTTP. If deployed separately, configure the MCP server URL with environment variables.
- The application should not require a paid database. Acceptable storage includes committed synthetic data files, SQLite, a small local vector store built during deployment, or a hosted free-tier option.
- If the free-tier service spins down after inactivity, the README and demo should explain the expected cold-start behavior.

8. CI/CD

- Create a GitHub Actions workflow, or equivalent CI/CD pipeline, that runs on push or pull request.
- The workflow should install dependencies, run a build/start/import check, and successfully execute some automated tests such as unit or smoke tests before deploying.
- Include at least one automated test that verifies the app can start and at least one test or script that verifies MCP tool discovery or a simple MCP tool call.
- Deployment must only occur if tests pass.

9. Evaluation of the Agentic RAG Application

- Provide an evaluation set of 20-30 questions or tasks covering policy Q&A and agentic workflows. Include straightforward policy questions, multi-document questions, tool-requiring tasks, ambiguous requests, and out-of-scope requests – with correct or gold answers.
- Report answer quality metrics: groundedness, citation accuracy, and optionally exact or partial match against short gold answers.
- Report agent behavior metrics: tool selection accuracy, workflow completion rate, escalation or clarification accuracy, and action-safety pass rate.
- Report system metrics: latency p50/p95 for 10-20 representative queries or tasks. If free-tier cold starts affect latency, report cold-start and warm-start behavior separately where possible.
- Include at least one ablation or comparison, such as different retrieval k values, chunk sizes, prompt variants, or agent tool availability.

10. Design Documentation

- Briefly justify design choices, including agent framework or manual orchestration approach, MCP server design, transport choice, tool schemas, embedding model, chunking strategy, retrieval k, vector store, deployment architecture, and safety guardrails.
- Include an architecture diagram or text-based architecture showing the web app, agent orchestrator, MCP client, MCP server or servers, RAG index, mock structured data, and LLM provider.
- Describe the two required agentic demo tasks and the expected sequence of MCP tool calls for each task.

Example Agentic Tasks

Students may choose their own workflows, but the deployed demo must include two completed agentic tasks. Examples include:

- **Remote work eligibility:** A user asks whether they can work remotely from another state or country for six weeks. The agent looks up the employee profile, retrieves remote work, security, tax/location, and approval policies, checks compliance, and produces cited next steps.
- **PTO request guidance:** A user asks whether they can take three days of PTO next week. The agent checks the employee's mock PTO balance, retrieves PTO policy, identifies manager approval requirements, and drafts a message or mock HR ticket if appropriate.
- **Expense compliance:** A user asks whether a laptop, home office chair, or travel expense can be reimbursed. The agent retrieves expense policy, checks employee role or location data if relevant, and returns a compliant decision with citations.
- **Benefits triage:** A user asks about eligibility for a benefit. The agent retrieves benefits policy, checks mock employment type and benefits status, and produces a grounded answer or escalation recommendation.
- **HR case triage:** A user describes a sensitive workplace issue. The agent retrieves relevant policies, determines whether the question should be answered directly or escalated, and creates a mock HR case summary if appropriate.

Recommended Free-Tier Architecture

The following architecture is recommended for students who want to remain within a typical free-tier deployment model:

```
Single Render/Railway Web Service
- Web chat UI and /chat API
- Agent orchestrator and MCP client
- Local MCP server process over stdio or localhost HTTP
- Policy RAG index using Chroma, FAISS, SQLite, or similar
- Mock employee, PTO, benefits, and ticket data as JSON or SQLite
- LLM and embedding providers configured through environment variables
```

Students are encouraged to use a more production-like architecture with separate MCP services, but this is not required. The project should be possible to complete without paid hosting, paid databases, or private organizational systems.

Submission Guidelines

Each student or group must submit two links: a [link to their presentation](#) and a [link to their GitHub repo](#). For a group, please ensure only **ONE** member submits on behalf of the group. Include the deployed application URL in your repository README.

Your submission must include both:

- 1) a link to your recorded demo presentation, and
 - 2) a link to your GitHub repository (either individual's repository or for a group, the group's shared repository)
-
1. A link to a recorded screen-share demonstration video of the working, deployed application, involving screen capture with voiceover.
 - The demo should be between 7 and 10 minutes long.
 - All group members must speak and be present on camera.
 - All group members must show their government ID.
 - The demo must show the system carrying out two agentic tasks end-to-end.

- For each agentic task, the presenter must explain how the agent is correctly calling MCP tools, including the tool names, arguments, outputs, retrieved citations, and final answer or action.
 - The demo should include a quick walkthrough of design, deployment, CI/CD, and evaluation results.
2. You must share your repository with the GitHub account, **quantic-grader**. Your GitHub repository must contain the following:
- All developed code
 - README.md with introductory description, setup, local run, deployment instructions.
 - design-and-evaluation.md explaining architecture, RAG design, MCP server design, agent orchestration, tool schemas, safety guardrails, deployment choices, and evaluation questions, expected answers and evaluation results.
 - ai-tooling.md describing which AI code tools you used and how, including what worked well and what did not.
 - deployed.md containing the deployed URL, health endpoint URL if available, and any notes about free-tier cold starts.
 - evaluation/ or equivalent containing the evaluation questions, expected answers or rubrics, scripts, and reported results.
 - mock_data/ or equivalent containing synthetic employee, PTO, benefits, and/or ticket data if used.
 - mcp/ or equivalent containing the MCP server code and tool definitions.

To submit your project, please click on the "Submit Project" button on your dashboard and follow the steps provided. If you are submitting your project as a group, please ensure only one member submits on behalf of the group. If you have chosen to work with a group on the web application, you will also be prompted to upload the final page of your [Group Project Agreement](#), which must be completed and signed by all group members. Please reach out to misaie+projects@quantic.edu if you have any questions. There is no score penalty for projects submitted after the due date, however grading may be delayed.

Plagiarism Policy

Here at Quantic, we believe that learning is best accomplished by doing. This ethos underpinned the design of our active learning platform, and it likewise informs our approach to the completion of projects and presentations for our degree programs. We expect that all of our graduates will be able to deploy the concepts and skills they have learned over the course of their degree, whether in the workplace or in pursuit of personal goals, and so it is in our students' best interest that these assignments be completed solely through their own efforts with academic integrity.

Quantic takes academic integrity very seriously. We define plagiarism as "Knowingly representing the work of others as one's own, engaging in any acts of plagiarism, or referencing the works of others without appropriate citation". This includes both misusing or not using proper citations for the works referenced, and submitting someone else's work as your own. When in doubt, **cite!** You can also find more about our plagiarism policy [here](#). Use of AI tools is permitted for the development of the agentic system in this project, but students must describe their use of AI tooling in ai-tooling.md and remain responsible for the correctness, security, and academic integrity of the submitted work.

Project Rubric

Scores 2 and above are considered passing. Students who receive a 1 or 0 will not get credit for the assignment and must revise and resubmit to receive a passing grade.

Score	Description
5	Addresses ALL of the project requirements at an outstanding level, including: • Outstanding deployed agentic HR system that combines policy RAG with correct, cited, grounded responses. • MCP integration is fully functional: the agent discovers and calls MCP-exposed tools correctly, with clear tool-call traces and graceful error handling. • At least two end-to-end agentic tasks are completed in the deployed demo, each requiring multi-step reasoning, RAG retrieval, and structured/mock-data tool use. • Excellent RAG ingestion, indexing, retrieval, citations, and guardrails. • Excellent architecture, including clear separation of web app, agent orchestration, MCP client/server, RAG index, mock data, and LLM provider. • Free-tier-compatible deployment on Render, Railway, or equivalent is fully functional, with environment variables and cold-start behavior documented. • CI/CD runs on push/PR and includes meaningful build/start checks plus at least one MCP tool discovery or call test. • Excellent evaluation results covering groundedness, citation accuracy, tool selection accuracy, workflow completion, safety, and latency. • Excellent design documentation and excellent demo presentation of features, architecture, deployment, MCP calls, and evaluation, meeting all demo requirements.
4	Addresses MOST of the project requirements at a very good level, including: • Very good deployed agentic HR system with generally correct, cited policy answers and functional workflows. • MCP integration works for the main tools, though traces, error handling, or architecture separation may have minor limitations. • Two agentic tasks are demonstrated, with mostly correct use of RAG, mock data, and MCP tools. • Very good ingestion, indexing, retrieval, citations, and guardrails. • Very good application architecture and deployment documentation. • Render, Railway, or equivalent deployment is mostly functional, with minor cold-start or configuration issues that do not prevent grading. • CI/CD runs on push/PR with useful build/start checks. • Very good evaluation covering most required RAG and agentic metrics. • Very good design documentation and demo presentation, meeting all demo requirements.

Score	Description
3	Addresses SOME of the project requirements at a good level, including: • Good RAG application with mainly correct answers and generally matching citations. • Agentic layer is present and can call at least some MCP-exposed tools, but tool use may be limited, inconsistently logged, or partly brittle. • At least one agentic task works well and a second is attempted, but one may be incomplete or require manual intervention. • Ingestion and indexing work, though retrieval quality or citation accuracy may be inconsistent. • Deployment is available or convincingly demonstrated with only moderate issues. • CI/CD runs with basic checks. • Evaluation includes most of groundedness, citation accuracy, latency, tool selection, and workflow completion, though depth may be limited. • Good documentation with some missing details, and a good demo meeting almost all demo requirements .
2	Addresses FEW of the project requirements at a passable level, including: • Passable RAG application with limited correct responses and limited or inconsistent citations. • MCP or agentic tool use is present but incomplete, shallow, or only partially functional. • Agentic tasks are attempted but not reliably completed end-to-end. • Ingestion, indexing, or retrieval works only partially. • Deployment is not fully functional, or local execution is required for substantial parts of the demo despite deployment being attempted. • CI/CD is present but minimal. • Evaluation includes only some required RAG or agentic metrics. • Documentation and demo presentation are passable but omit important details.
1	Addresses the project but MOST requirements are missing, including: • Incomplete app or app cannot be run by the grader. • No meaningful MCP integration or no working agentic workflow. • No or very limited RAG functionality. • No CI/CD or no meaningful deployment attempt. • No or very limited evaluation. • No meaningful design documentation. • No usable demo presentation of the application.
0	The student either did not complete the assignment, plagiarized all or part of the assignment, or completely failed to address the project requirements .